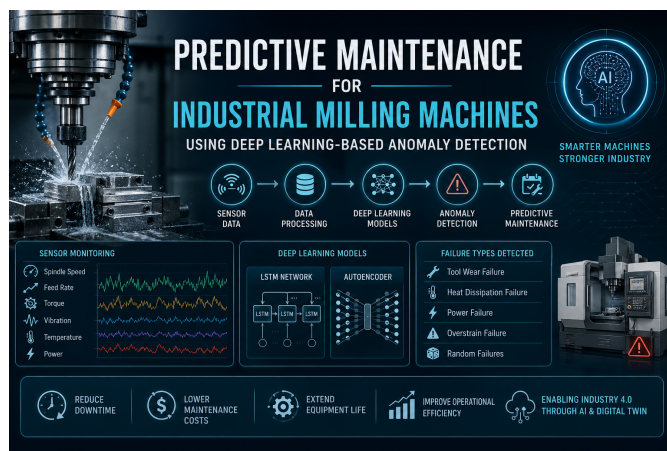


IIT Madras

A project report submitted for DA6401W Deep Learning



Predictive Maintenance for Industrial Milling Machines Using Deep Learning-Based Anomaly Detection

by **Amardeep Kumar**

April 30, 2026

IIT Madras

ABSTRACT

A project report submitted for DA6401W Deep Learning

by Amardeep Kumar

This project presents a deep learning-based predictive maintenance system for industrial milling machines aimed at predicting equipment failures before they occur. The study utilizes the AI4I 2020 predictive maintenance dataset, which contains real-world sensor data and multiple failure modes.

The proposed approach integrates multiple models, including a Multi-Layer Perceptron (MLP) for baseline classification, a Long Short-Term Memory (LSTM) network to capture temporal dependencies, and an Autoencoder for anomaly detection. These models enable the identification of five distinct failure types: tool wear failure, heat dissipation failure, power failure, overstrain failure, and random failures.

Experimental results demonstrate the effectiveness of deep learning techniques in accurately predicting failures and detecting anomalies. Additionally, Weights & Biases (WandB) is used for experiment tracking, enabling performance monitoring and model comparison.

By implementing this predictive framework, manufacturing systems can transition from reactive to proactive maintenance strategies, reducing unplanned downtime and improving operational efficiency. This work highlights the practical application of deep learning in Industry 4.0 and Digital Twin environments.

Contents

1	Introduction	1
2	Background and State-of-the-art	3
3	Problem Statement	5
4	Dataset Description	7
4.1	Dataset Overview	7
4.2	Exploratory Data Analysis	8
4.2.1	Machine Failure Across Machine Types	8
4.2.2	Feature Distribution and Outlier Analysis	9
4.2.3	Feature Relationships and Pairwise Analysis	9
4.2.4	Correlation Analysis	9
4.2.5	Summary of Insights	10
5	Solution/Methodology/Neural Net Architecture	11
5.1	Multi-Layer Perceptron (MLP)	11
5.2	Long Short-Term Memory (LSTM)	12
5.3	Autoencoder for Anomaly Detection	13
5.3.1	Architecture	13
5.3.2	Reconstruction Error	14
5.3.3	Anomaly Detection	15
6	Theoretical Analysis	16
7	Data Preprocessing and Training Methodology	17
7.1	Data Preprocessing	17
7.1.1	Data Cleaning	17
7.1.2	Encoding : Label Encoding	17
7.1.3	Feature Scaling : Z-Score Normalization	18
7.1.4	Train-Test Split	18
7.1.5	Data Preparation for Training	18
7.2	Training Methodology	19
7.2.1	Loss Functions	19
7.2.2	Optimizer	19
7.2.3	Training Loop	20

7.2.4	Autoencoder Training Strategy	20
7.2.5	WandB	21
8	Experimental Results	22
8.1	MLP Model Performance	22
8.1.1	Training Performance	22
8.1.2	Classification Report	23
8.1.3	Confusion Matrix	23
8.1.4	ROC Curve	23
8.2	LSTM Model Performance	24
8.2.1	Training Performance	24
8.2.2	Classification Report	25
8.2.3	Confusion Matrix	25
8.3	Autoencoder Model Performance	26
8.3.1	Training Performance	26
8.3.2	Classification Report	26
8.3.3	Confusion Matrix	27
8.3.4	Reconstruction Error Analysis	27
8.4	Experiment Autoencoder Model Performance	28
8.4.1	Training Performance	28
8.4.2	Result Comparison Across Experiments	28
8.5	Comparative Analysis	29
8.6	Key Observations	30
8.7	Discussion and Future Improvements	31
9	Inferences/Insights Obtained/Difficulties faced	32
9.1	Model Performance Analysis	32
9.2	Analysis of LSTM Performance	32
9.3	Key Insights	33
9.4	Difficulties Faced	33
9.4.1	Class Imbalance	33
9.4.2	Sequence Generation for LSTM	33
9.4.3	Model Selection	33
9.5	Conclusion	34
10	Implementation Details and Resources	35

List of Figures

4.1	Dataset Summary	8
4.2	Machine failure distribution across machine types	8
4.3	Outlier analysis for rotational speed and torque	9
4.4	Pairwise relationships between features	9
4.5	Correlation matrix of features	10
5.1	Architecture of the MLP model	12
5.2	Architecture of the LSTM model	13
5.3	Architecture of the LSTM model	14
8.1	Training loss MLP across epochs (WandB visualization)	22
8.2	Classification report for the MLP model	23
8.3	Confusion matrix for the MLP model	23
8.4	ROC curve for the MLP model	24
8.5	Training loss LSTM across epochs (WandB visualization)	24
8.6	Classification report for the LSTM model	25
8.7	Confusion matrix for the LSTM model	25
8.8	Training loss Autoencoder across epochs (WandB visualization)	26
8.9	Classification report for the Autoencoder model	26
8.10	Confusion matrix for the Autoencoder model	27
8.11	Reconstruction error distribution with anomaly threshold	27
8.12	Training loss comparison across multiple Autoencoder experiments with varying hyperparameters	28
8.13	Comparison of evaluation metrics (ROC-AUC, Recall, Precision, F1-score, Threshold) across different Autoencoder experiments	29

List of Tables

8.1	Comparison of model performance	30
-----	---	----

Chapter 1

Introduction

In modern manufacturing environments, unplanned equipment failures can lead to significant financial losses, production delays, and reduced operational efficiency. Traditional maintenance strategies, such as reactive and preventive maintenance, often fail to optimally address these challenges.

With the rise of Industry 4.0, the integration of sensors and data-driven technologies has enabled the development of predictive maintenance systems. These systems leverage historical and real-time data to anticipate failures before they occur, allowing organizations to take proactive actions.

However, accurately predicting failures in complex industrial systems remains a challenging task due to nonlinear patterns and high-dimensional sensor data. Conventional machine learning techniques often struggle to capture temporal dependencies and subtle anomalies in such data.

To address these challenges, this project proposes a deep learning-based predictive maintenance framework using LSTM networks and Autoencoder models. The LSTM model captures temporal dependencies in sensor data, while the Autoencoder detects anomalies by learning normal operational patterns.

The main objectives of this project are:

- To develop a classification model for predicting machine failures.
- To implement an anomaly detection system using Autoencoders.
- To analyze multiple failure modes in industrial milling machines.

The rest of the report is organized as follows: Chapter 2 discusses background and related work, Chapter 3 defines the problem statement, Chapter 4 describes the dataset, and subsequent chapters present the methodology, experiments, and results.

Chapter 2

Background and State-of-the-art

Predictive maintenance has emerged as a critical application in modern manufacturing systems, particularly within the framework of Industry 4.0. Industry 4.0 emphasizes the integration of cyber-physical systems, Internet of Things (IoT), and real-time data analytics to enable intelligent and autonomous industrial operations. In such environments, machines are equipped with sensors that continuously generate data, enabling advanced monitoring and decision-making.

A closely related concept is the Digital Twin, which refers to a virtual representation of a physical asset that is continuously updated using real-time sensor data. Digital Twins enable real-time monitoring, simulation, and predictive analysis of machine behavior. In the context of predictive maintenance, Digital Twins leverage data-driven models to detect anomalies, forecast failures, and optimize maintenance scheduling.

Traditional maintenance strategies, such as reactive and preventive maintenance, are either inefficient or suboptimal, as they fail to leverage real-time data. Early predictive maintenance systems relied on statistical methods and classical machine learning algorithms, including decision trees and support vector machines. However, these approaches often struggle with high-dimensional and nonlinear sensor data.

Recent advancements in deep learning have significantly improved the performance of predictive maintenance systems. Multi-Layer Perceptrons (MLP) are widely used for classification tasks, while Long Short-Term Memory (LSTM) networks are capable of capturing temporal dependencies in sequential data. Furthermore, Autoencoder architectures have gained popularity for anomaly detection, as they

can learn the normal operating conditions of machines and identify deviations effectively.

These developments form the foundation for the proposed deep learning-based predictive maintenance system, which aligns with Industry 4.0 principles and contributes toward Digital Twin-enabled intelligent maintenance frameworks.

Chapter 3

Problem Statement

In industrial manufacturing systems, machines continuously generate large volumes of sensor data reflecting their operational conditions. Accurately predicting equipment failures from such data is critical for minimizing downtime and reducing maintenance costs. However, this task is inherently challenging due to several factors.

First, the relationship between sensor measurements and machine failure is highly non-linear and influenced by multiple interacting variables such as temperature, torque, and rotational speed. Traditional machine learning models often struggle to capture these complex dependencies effectively.

Second, the dataset contains multiple underlying failure modes (tool wear, heat dissipation, power failure, overstrain, and random failures), which collectively determine the final failure label. This makes the prediction problem more complex, as the model must implicitly learn these hidden relationships.

Third, the dataset is highly imbalanced, with failure cases forming a small minority compared to normal operating conditions. This leads to biased models that achieve high accuracy but fail to correctly identify critical failure events.

Furthermore, the dataset does not explicitly contain temporal sequences, making it challenging to apply sequence-based models such as LSTM without additional preprocessing.

Therefore, the objective of this project is to develop a robust predictive maintenance system that can:

- Accurately classify machine failure using sensor data

- Detect anomalous machine behavior using unsupervised learning
- Handle class imbalance effectively
- Evaluate the suitability of different deep learning architectures (MLP, LSTM, Autoencoder)

To address these challenges, this project addresses predictive maintenance using both supervised classification and unsupervised anomaly detection approaches. While classification models (MLP, LSTM) attempt to directly predict failure labels, the Autoencoder learns normal operating behavior and identifies deviations as anomalies. This dual approach enables comparison between label-driven prediction and behavior-driven detection, especially under class imbalance conditions.

Chapter 4

Dataset Description

4.1 Dataset Overview

The dataset used in this project is the AI4I 2020 Predictive Maintenance dataset [1], which is publicly available on Kaggle. It consists of 10,000 observations representing simulated sensor data from industrial milling machines.

Each data instance includes multiple features such as air temperature, process temperature, rotational speed, torque, and tool wear. Additionally, categorical information such as product type (L, M, H) is provided to represent different quality variants.

The target variable, *Machine failure*, is a binary indicator that denotes whether a failure has occurred. This label is derived from multiple underlying failure modes, including:

- Tool Wear Failure (TWF)
- Heat Dissipation Failure (HDF)
- Power Failure (PWF)
- Overstrain Failure (OSF)
- Random Failure (RNF)

Each failure mode is triggered based on specific conditions involving sensor readings and operational thresholds. The overall machine failure label is set to 1 if any of these failure conditions are met.

This dataset provides a realistic representation of industrial machine behavior, capturing both normal operating conditions and various failure scenarios. It is well-suited for supervised classification as well as anomaly detection tasks in predictive maintenance applications.

	dtypes	count	#unique	#missing	missig%	mean	std	min	25%	50%	75%	max
UDI	int64	10000	10000	0	0.000000	5000.500000	2886.895680	1.000000	2500.750000	5000.500000	7500.250000	10000.000000
Product ID	object	10000	10000	0	0.000000	nan	nan	nan	nan	nan	nan	nan
Type	object	10000	3	0	0.000000	nan	nan	nan	nan	nan	nan	nan
Air temperature [K]	float64	10000	93	0	0.000000	300.004930	2.000259	295.300000	298.300000	300.100000	301.500000	304.500000
Process temperature [K]	float64	10000	82	0	0.000000	310.005560	1.483734	305.700000	308.800000	310.100000	311.100000	313.800000
Rotational speed [rpm]	int64	10000	941	0	0.000000	1538.776100	179.284096	1168.000000	1423.000000	1503.000000	1612.000000	2886.000000
Torque [Nm]	float64	10000	577	0	0.000000	39.986910	9.968934	3.800000	33.200000	40.100000	46.800000	76.600000
Tool wear [min]	int64	10000	246	0	0.000000	107.951000	63.654147	0.000000	53.000000	108.000000	162.000000	253.000000
Machine failure	int64	10000	2	0	0.000000	0.033900	0.180981	0.000000	0.000000	0.000000	0.000000	1.000000
TWF	int64	10000	2	0	0.000000	0.004600	0.067671	0.000000	0.000000	0.000000	0.000000	1.000000
HDF	int64	10000	2	0	0.000000	0.011500	0.106625	0.000000	0.000000	0.000000	0.000000	1.000000
PWF	int64	10000	2	0	0.000000	0.009500	0.097009	0.000000	0.000000	0.000000	0.000000	1.000000
OSF	int64	10000	2	0	0.000000	0.009800	0.098514	0.000000	0.000000	0.000000	0.000000	1.000000
RNF	int64	10000	2	0	0.000000	0.001900	0.043550	0.000000	0.000000	0.000000	0.000000	1.000000

FIGURE 4.1: Dataset Summary

4.2 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to understand the underlying structure of the dataset, identify patterns, detect anomalies, and establish relationships between features. The analysis also helps in validating assumptions and guiding model selection.

4.2.1 Machine Failure Across Machine Types

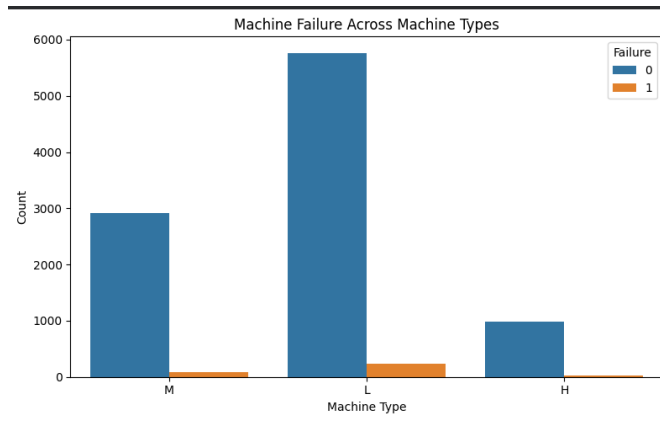


FIGURE 4.2: Machine failure distribution across machine types

The analysis shows that machines of type L (Low quality variant) exhibit the highest number of failures compared to types M and H. This indicates that machine

type plays a significant role in failure occurrence, possibly due to differences in operational load or quality characteristics.

4.2.2 Feature Distribution and Outlier Analysis

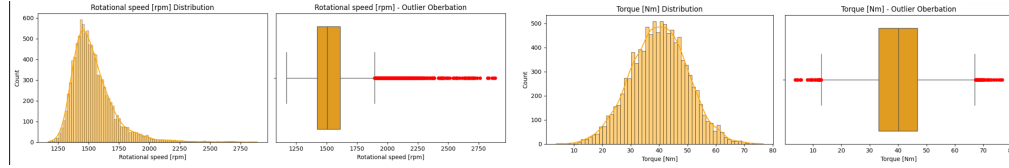


FIGURE 4.3: Outlier analysis for rotational speed and torque

Boxplot analysis reveals the presence of significant outliers in features such as rotational speed and torque. These extreme values may correspond to abnormal machine behavior or rare operating conditions, which are critical for anomaly detection tasks.

4.2.3 Feature Relationships and Pairwise Analysis

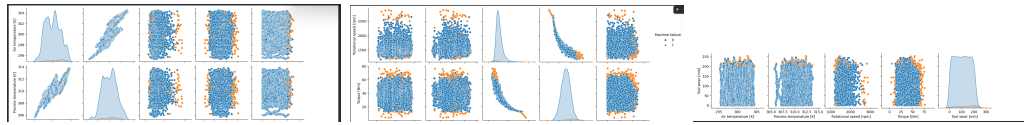


FIGURE 4.4: Pairwise relationships between features

Pairwise analysis highlights the relationships between different features and provides insight into how they interact with each other. Although some separation between failure and non-failure cases is observed, the relationships are largely non-linear, justifying the use of deep learning models such as MLP and LSTM.

4.2.4 Correlation Analysis

Correlation analysis reveals several important relationships:

- Air temperature and process temperature exhibit a strong positive correlation.
- Rotational speed and torque show a negative correlation, indicating an inverse relationship.

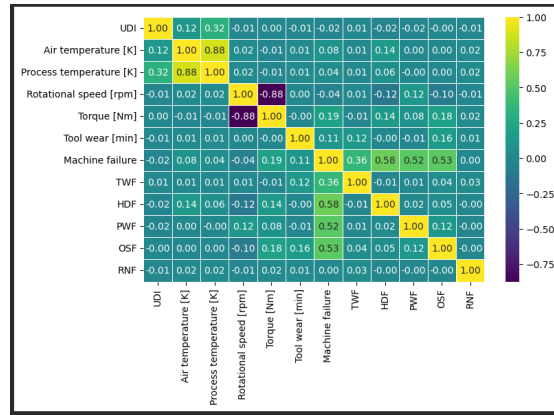


FIGURE 4.5: Correlation matrix of features

- Machine failure is strongly correlated with specific failure modes such as tool wear failure (TWF), heat dissipation failure (HDF), power failure (PWF), overstrain failure (OSF), and random failure (RNF).

These insights help in understanding feature dependencies and play a crucial role in model selection and feature engineering.

4.2.5 Summary of Insights

- Machine type significantly influences failure occurrence.
- Presence of outliers suggests abnormal operating conditions, supporting anomaly detection approaches.
- Strong correlations exist among temperature-related features.
- Non-linear relationships between features justify the use of deep learning models.

Chapter 5

Solution/Methodology/Neural Net Architecture

This project employs deep learning techniques to address predictive maintenance through both classification and anomaly detection approaches. Three models were implemented: a Multi-Layer Perceptron (MLP), a Long Short-Term Memory (LSTM) network, and an Autoencoder.

The MLP and LSTM models are used for supervised classification of machine failures, while the Autoencoder is used for unsupervised anomaly detection.

5.1 Multi-Layer Perceptron (MLP)

The Multi-Layer Perceptron (MLP) is a feedforward neural network used for binary classification of machine failure. It learns complex non-linear relationships between input features through multiple fully connected layers.

- **Input Layer:** Takes normalized feature vectors representing machine sensor data.
- **Hidden Layers:** Two fully connected dense layers with 64 and 32 neurons respectively, each followed by ReLU activation to introduce non-linearity.
- **Output Layer:** A fully connected layer with 2 output neurons corresponding to the two classes (failure and non-failure). The outputs are raw logits,

and the Softmax function is applied implicitly during training via the loss function.

The model is trained using the Cross-Entropy loss function, which internally applies the Softmax operation to compute class probabilities.

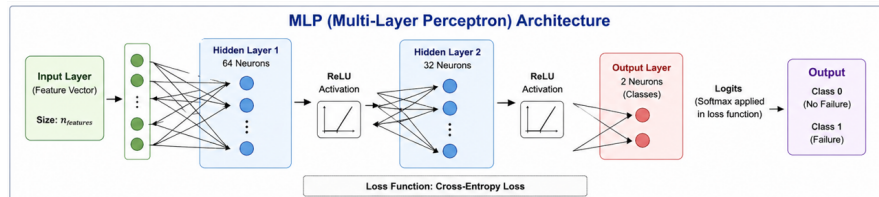


FIGURE 5.1: Architecture of the MLP model

```
class MLP(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 2)
        )

    def forward(self, x):
        return self.net(x)
```

LISTING 5.1: MLP Architecture

5.2 Long Short-Term Memory (LSTM)

The Long Short-Term Memory (LSTM) network is a type of recurrent neural network (RNN) designed to capture temporal dependencies in sequential data. It is particularly effective in modeling long-term relationships by addressing the vanishing gradient problem present in traditional RNNs.

In this project, since the dataset is not inherently sequential, a sliding window approach was used to transform the tabular data into sequential format. Each input sample consists of a fixed-length sequence of feature vectors.

- **Input:** Sequential feature data of shape (batch size, sequence length, input dimension).
- **LSTM Layer:** A single LSTM layer with 64 hidden units processes the input sequence and captures temporal patterns.

- **Hidden State:** The final hidden state of the LSTM is used as a representation of the sequence.
- **Fully Connected Layer:** A dense layer maps the hidden representation to 2 output neurons corresponding to failure and non-failure classes.

The model outputs raw logits, and the Softmax operation is applied implicitly through the Cross-Entropy loss function during training.

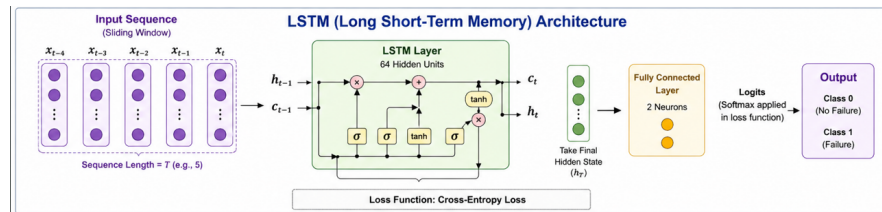


FIGURE 5.2: Architecture of the LSTM model

```
class LSTMModel(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, 64, batch_first=True)
        self.fc = nn.Linear(64, 2)

    def forward(self, x):
        _, (hn, _) = self.lstm(x)
        return self.fc(hn[-1])
```

LISTING 5.2: LSTM Architecture

5.3 Autoencoder for Anomaly Detection

The Autoencoder is an unsupervised deep learning model used for anomaly detection. It learns to reconstruct normal data patterns and identifies anomalies based on reconstruction error.

5.3.1 Architecture

The Autoencoder consists of two main components:

- **Encoder:** Compresses the input feature vector into a lower-dimensional latent representation.

- **Decoder:** Reconstructs the original input from the compressed representation.
- **Input Layer:** Takes normalized feature vectors
- **Encoder Layers:** Fully connected layers (input $\rightarrow 32 \rightarrow 16$)
- **Bottleneck Layer:** 16-dimensional compressed representation
- **Decoder Layers:** Fully connected layers (16 $\rightarrow 32 \rightarrow$ input dimension)

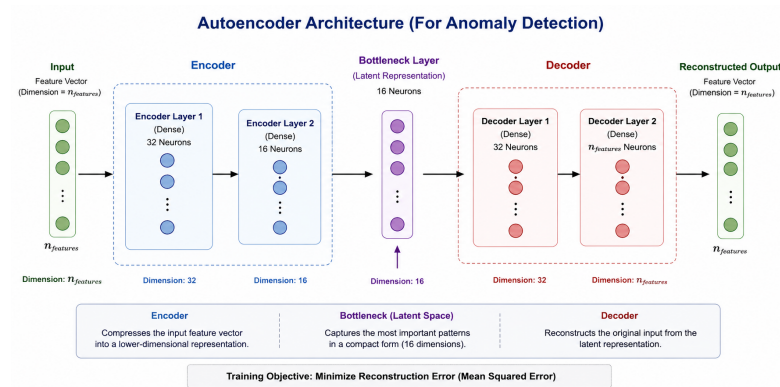


FIGURE 5.3: Architecture of the LSTM model

```
class Autoencoder(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 16)
        )
        self.decoder = nn.Sequential(
            nn.Linear(16, 32),
            nn.ReLU(),
            nn.Linear(32, input_dim)
        )

    def forward(self, x):
        encoded = self.encoder(x)
        return self.decoder(encoded)
```

LISTING 5.3: Autoencoder Architecture

5.3.2 Reconstruction Error

Anomaly detection is based on reconstruction error, defined as:

$$\text{Reconstruction Error} = ||X - \hat{X}||^2 \quad (5.1)$$

where X is the input and $\hat{X}$ is the reconstructed output.

Normal data points have low reconstruction error, while anomalous data points have high reconstruction error.

5.3.3 Anomaly Detection

During inference, the reconstruction error is computed for each sample. A threshold is defined as:

$$\text{Threshold} = \mu + 2\sigma \quad (5.2)$$

where μ is the mean reconstruction error and σ is the standard deviation.

Samples with reconstruction error greater than the threshold are classified as anomalies.

Chapter 6

Theoretical Analysis

This project employs multiple deep learning architectures to address both classification and anomaly detection tasks.

The Multi-Layer Perceptron (MLP) is a feedforward neural network that consists of multiple fully connected layers. It is used for supervised classification of machine failures based on sensor data.

Long Short-Term Memory (LSTM) networks are a type of recurrent neural network designed to capture temporal dependencies in sequential data. They are particularly useful for modeling time-series patterns in industrial sensor data.

Autoencoders are unsupervised neural networks that learn to reconstruct input data. By training on normal operating conditions, they can detect anomalies by measuring reconstruction error, where higher errors indicate deviations from normal behavior.

The integration of these models enables the system to perform both predictive failure classification and anomaly detection, providing a comprehensive predictive maintenance solution.

Chapter 7

Data Preprocessing and Training Methodology

7.1 Data Preprocessing

Data preprocessing is a crucial step to ensure data quality and improve model performance. The following preprocessing steps were applied to the dataset.

7.1.1 Data Cleaning

Irrelevant columns such as *UDI* and *Product ID* were removed from the dataset, as they serve only as identifiers and do not contribute to predictive modeling.

Outlier analysis revealed extreme values in features such as rotational speed and torque. These values were retained as they represent abnormal operating conditions and are important for predictive maintenance and anomaly detection tasks.

7.1.2 Encoding : Label Encoding

The categorical feature *Type* (L, M, H) was converted into numerical form using label encoding to make it suitable for machine learning models.

```
encoder = LabelEncoder()  
data['Type'] = encoder.fit_transform(data['Type'])
```

LISTING 7.1: Label Encoding

7.1.3 Feature Scaling : Z-Score Normalization

Z-score normalization (standardization) was applied to scale the numerical features. It is defined as:

$$z = \frac{x - \mu}{\sigma} \quad (7.1)$$

where x is the original feature value, μ is the mean of the feature, and σ is the standard deviation. This transformation ensures that the features have zero mean and unit variance, which improves convergence and stabilizes the training of deep learning models.

```
X = data.drop('Machine failure', axis=1)
y = data['Machine failure']
```

LISTING 7.2: Feature and Target Split

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

LISTING 7.3: Z-score Normalization using StandardScaler

7.1.4 Train-Test Split

The dataset was divided into training and testing sets using an 80:20 ratio. The training set was used to train the models, while the test set was used to evaluate model performance on unseen data.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

LISTING 7.4: Train-Test Split

7.1.5 Data Preparation for Training

The processed data was converted into PyTorch tensors and loaded using DataLoader. This enables efficient batching, shuffling, and parallel data loading during model training.

```
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train.values, dtype=torch.long)
```

LISTING 7.5: Conversion to PyTorch Tensors

7.2 Training Methodology

The models were trained using supervised and unsupervised learning approaches depending on the task. The training process involves forward propagation, loss computation, backpropagation, and weight updates using an optimizer.

7.2.1 Loss Functions

Cross-Entropy Loss was used for classification tasks, while Mean Squared Error (MSE) was used for training the Autoencoder.

7.2.2 Optimizer

The Adam (Adaptive Moment Estimation) optimizer was used for training the models due to its efficiency and fast convergence properties. Adam combines the advantages of two widely used optimization techniques: Momentum and RMSProp.

Adam maintains two moving averages for each parameter:

- The first moment (mean of gradients)
- The second moment (uncentered variance of gradients)

The update rules for Adam are defined as follows:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (7.2)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (7.3)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (7.4)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (7.5)$$

where:

- g_t is the gradient at time step t
- m_t and v_t are the first and second moment estimates
- β_1 and β_2 are decay rates (commonly 0.9 and 0.999)
- α is the learning rate
- ϵ is a small constant to prevent division by zero

Adam is particularly well-suited for deep learning models such as MLP, LSTM, and Autoencoders, as it adapts the learning rate for each parameter and handles sparse gradients efficiently, leading to faster and more stable convergence.

7.2.3 Training Loop

```
for epoch in range(epochs):
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
```

LISTING 7.6: Training Loop

7.2.4 Autoencoder Training Strategy

The Autoencoder was trained only on normal (non-failure) data to learn the underlying distribution of healthy machine behavior.

- Training Data: Samples where machine failure = 0
- Loss Function: Mean Squared Error (MSE)
- Optimizer: Adam

The objective is to minimize the reconstruction error between the input and the reconstructed output.

7.2.5 WandB

Weights and Biases (WandB) was used for experiment tracking, including monitoring loss, metrics, and model performance across epochs.

```
from wandb_utils import WandBLogger

w_logger = WandBLogger(
    project="Predictive Maintenance for Industrial Milling Machines",
    config={"model": "MLP", "epochs": 10, "lr": 0.001}
)

w_logger.watch(mlp_model)

for epoch in range(10):
    mlp_model.train()

    outputs = mlp_model(X_train)
    loss = criterion(outputs, y_train)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    w_logger.log({"loss": loss.item()})

w_logger.finish()
```

LISTING 7.7: WandB Integration for Experiment Tracking

Chapter 8

Experimental Results

8.1 MLP Model Performance

8.1.1 Training Performance

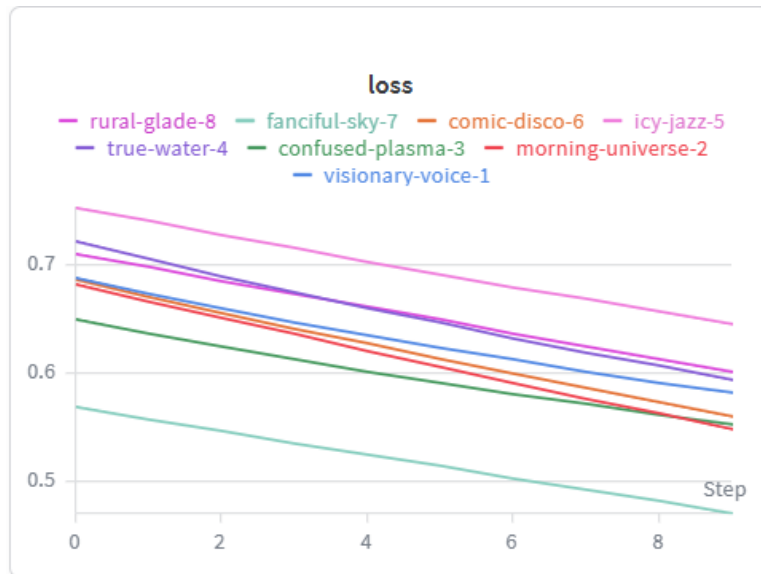


FIGURE 8.1: Training loss MLP across epochs (WandB visualization)

The training loss curve shows a steady decrease over epochs, indicating that the models are able to learn meaningful patterns from the data. The smooth decline suggests stable convergence without significant oscillations.

8.1.2 Classification Report

The MLP model achieves high overall accuracy; however, the performance is heavily skewed toward the majority class.

```
*** Accuracy: 0.7785
```

	precision	recall	f1-score	support
0	0.99	0.78	0.87	1939
1	0.08	0.62	0.15	61
accuracy			0.78	2000
macro avg	0.53	0.70	0.51	2000
weighted avg	0.96	0.78	0.85	2000

FIGURE 8.2: Classification report for the MLP model

8.1.3 Confusion Matrix

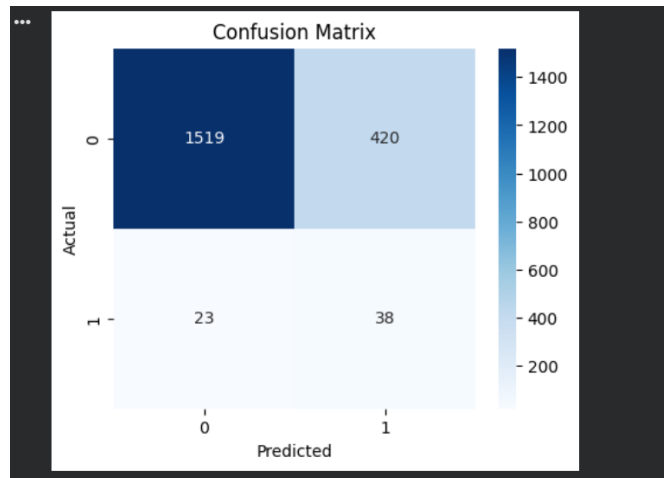


FIGURE 8.3: Confusion matrix for the MLP model

The confusion matrix reveals that the model correctly classifies most non-failure instances but fails to identify failure cases. A large number of false negatives indicates poor recall for the minority class.

8.1.4 ROC Curve

The ROC-AUC score is observed to be significantly low, indicating poor discriminative ability. This suggests that the model is unable to effectively distinguish between failure and non-failure cases due to class imbalance.

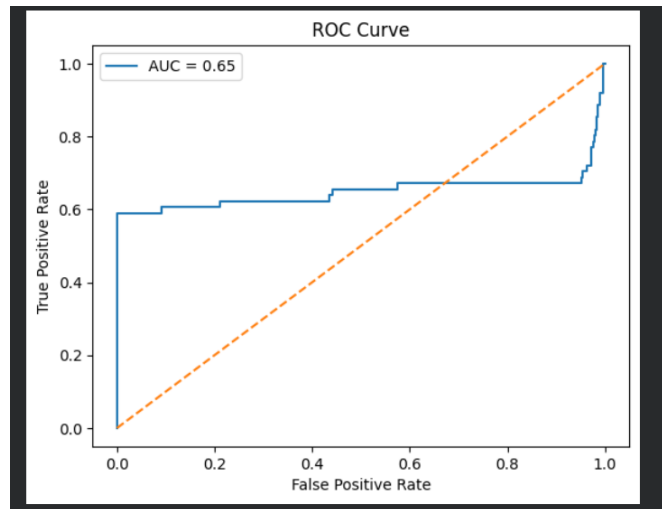


FIGURE 8.4: ROC curve for the MLP model

8.2 LSTM Model Performance

8.2.1 Training Performance



FIGURE 8.5: Training loss LSTM across epochs (WandB visualization)

The training loss curve shows a steady decrease over epochs, indicating that the models are able to learn meaningful patterns from the data. The smooth decline suggests stable convergence without significant oscillations.

8.2.2 Classification Report

The LSTM model shows similar performance trends to the MLP model, with high accuracy but poor recall for failure cases.

Accuracy: 0.968984492246123				
	precision	recall	f1-score	support
0	0.97	1.00	0.98	1937
1	0.00	0.00	0.00	62
accuracy			0.97	1999
macro avg	0.48	0.50	0.49	1999
weighted avg	0.94	0.97	0.95	1999

FIGURE 8.6: Classification report for the LSTM model

8.2.3 Confusion Matrix

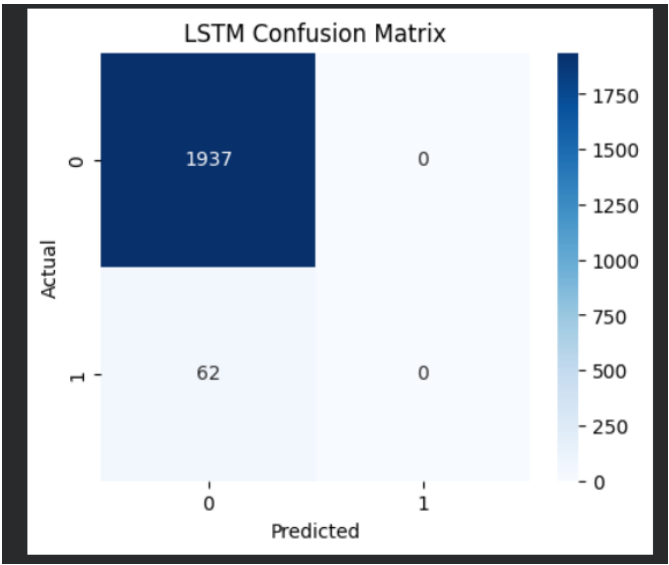


FIGURE 8.7: Confusion matrix for the LSTM model

The confusion matrix highlights that the LSTM model also struggles to correctly classify failure instances, reinforcing the impact of class imbalance.

8.3 Autoencoder Model Performance

8.3.1 Training Performance

The training loss curve shows a steady decrease over epochs, indicating that the models are able to learn meaningful patterns from the data. The smooth decline suggests stable convergence without significant oscillations.

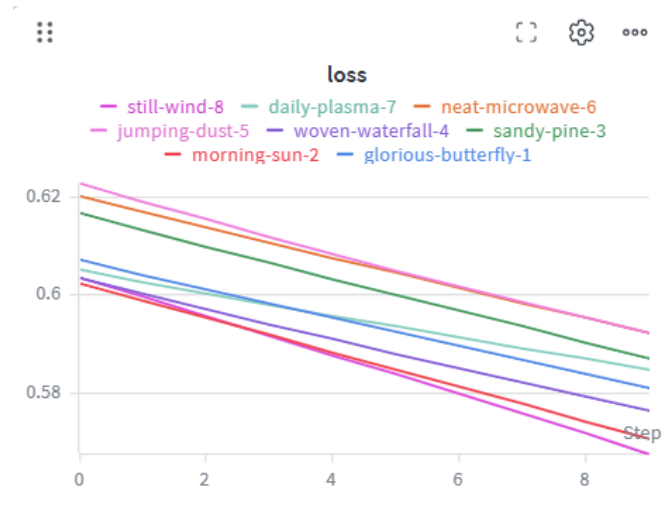


FIGURE 8.8: Training loss Autoencoder across epochs (WandB visualization)

8.3.2 Classification Report

Anomaly Detection Done					
	precision	recall	f1-score	support	
0	1.00	1.00	1.00	1939	
1	0.91	0.97	0.94	61	
accuracy			1.00	2000	
macro avg	0.95	0.98	0.97	2000	
weighted avg	1.00	1.00	1.00	2000	
ROC-AUC: 0.9788381707657319					

FIGURE 8.9: Classification report for the Autoencoder model

The Autoencoder achieves strong performance across both classes. Unlike the MLP model, it is able to effectively detect failure cases with high precision and recall. This demonstrates its ability to identify anomalies despite the presence of class imbalance.

8.3.3 Confusion Matrix

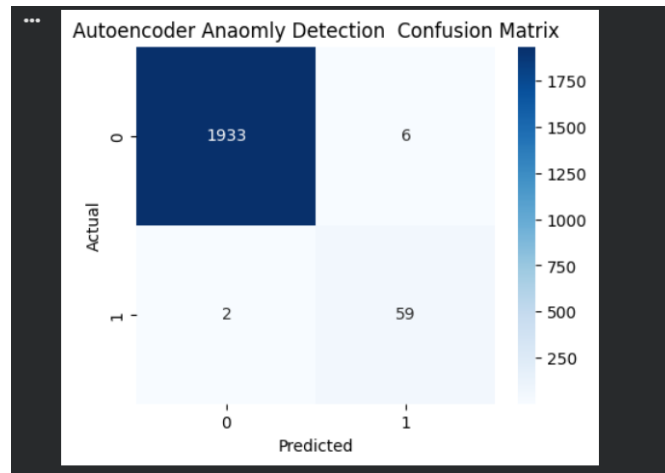


FIGURE 8.10: Confusion matrix for the Autoencoder model

The confusion matrix shows that the Autoencoder correctly classifies the majority of both normal and failure instances. The number of false positives and false negatives is very low, indicating reliable anomaly detection performance.

8.3.4 Reconstruction Error Analysis

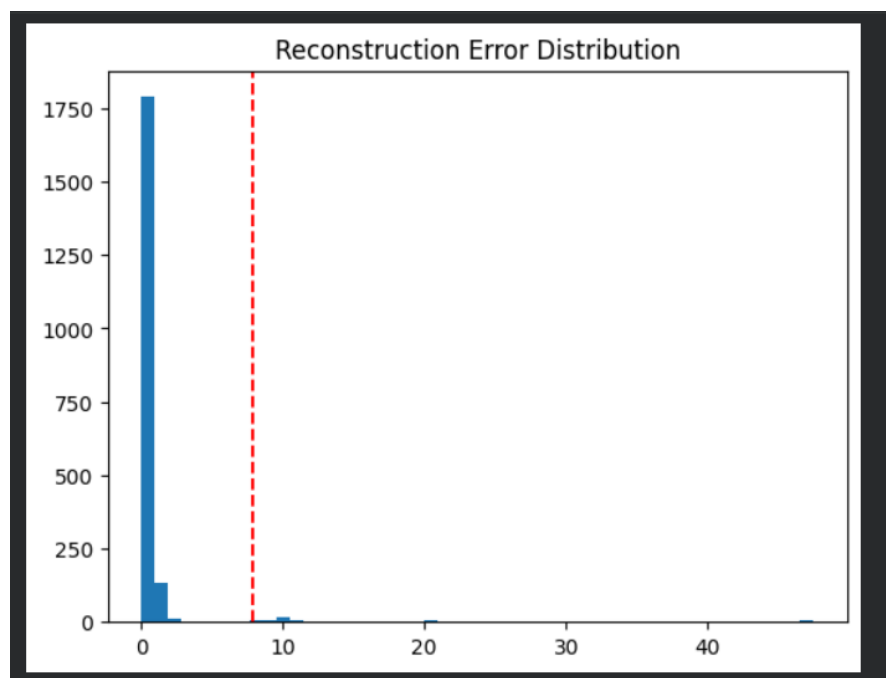


FIGURE 8.11: Reconstruction error distribution with anomaly threshold

The reconstruction error distribution highlights a clear separation between normal and anomalous samples. Most normal data points have low reconstruction error,

while failure cases exhibit higher error values. The selected threshold effectively distinguishes between normal and abnormal behavior.

8.4 Experiment Autoencoder Model Performance

8.4.1 Training Performance

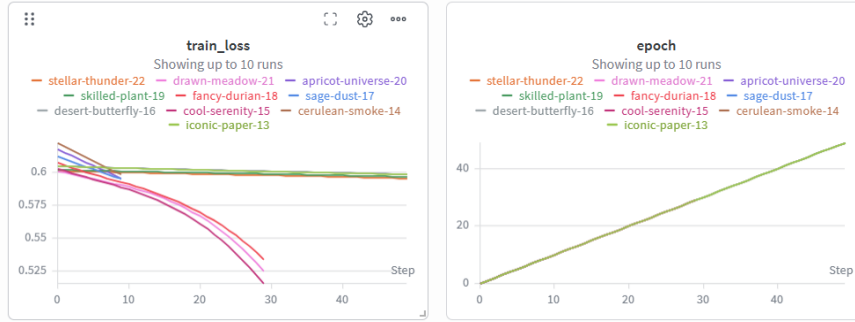


FIGURE 8.12: Training loss comparison across multiple Autoencoder experiments with varying hyperparameters

The training loss curves demonstrate the impact of different hyperparameter configurations on model convergence. Experiments with higher learning rates and optimal network depth show faster convergence and lower final loss values.

Some configurations exhibit slower convergence, indicating suboptimal learning rates or insufficient model capacity. Overall, the results highlight the sensitivity of Autoencoder performance to hyperparameter tuning, particularly learning rate and number of layers.

8.4.2 Result Comparison Across Experiments

The experimental results compare multiple Autoencoder configurations based on key evaluation metrics such as ROC-AUC, Recall, Precision, and F1-score.

It is observed that:

- Certain configurations achieve consistently higher ROC-AUC scores, indicating better discrimination between normal and anomalous samples.
- Recall remains high across most experiments, which is critical for predictive maintenance as missing a failure can lead to severe consequences.

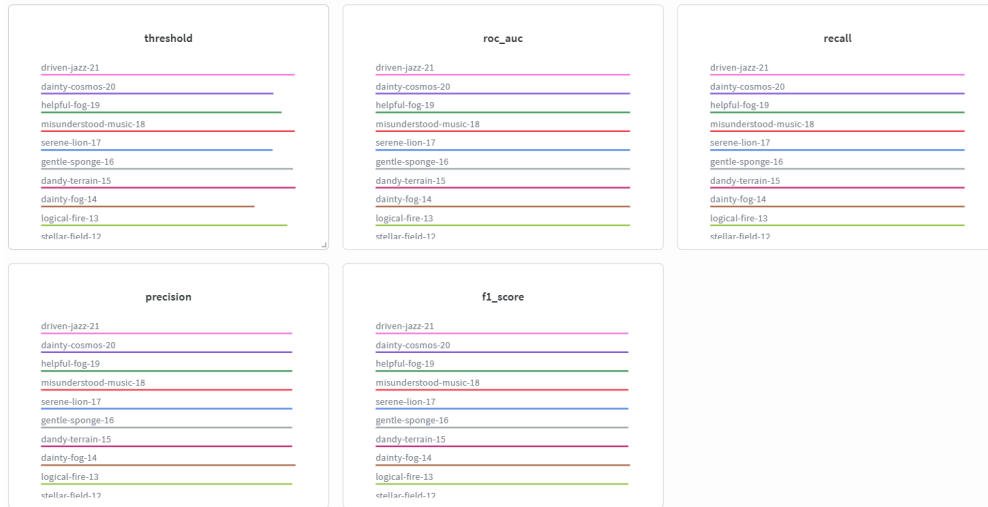


FIGURE 8.13: Comparison of evaluation metrics (ROC-AUC, Recall, Precision, F1-score, Threshold) across different Autoencoder experiments

- Precision varies slightly, suggesting a trade-off between detecting anomalies and avoiding false positives.
- F1-score provides a balanced measure and highlights the best-performing configurations.
- Threshold values differ across runs, reflecting variations in reconstruction error distributions.

Overall, the experiment demonstrates that hyperparameter tuning significantly improves anomaly detection performance, with some configurations clearly outperforming others.

8.5 Comparative Analysis

The performance of MLP, LSTM, and Autoencoder models reveals significant differences in their ability to handle imbalanced data and detect failure cases.

- **MLP Model:** Although the model achieves high overall accuracy, it fails to detect failure cases effectively. The confusion matrix and ROC-AUC score indicate poor recall and weak discriminative ability, primarily due to class imbalance.

- **LSTM Model:** The LSTM model exhibits similar behavior to the MLP model. Despite its capability to model sequential data, it does not show significant improvement. This is likely because the dataset lacks strong temporal dependencies, limiting the effectiveness of sequence modeling.
- **Autoencoder Model:** The Autoencoder significantly outperforms both MLP and LSTM in detecting anomalies. It achieves high precision and recall for failure cases, with very few false positives and false negatives. The reconstruction error distribution clearly separates normal and anomalous samples, enabling effective threshold-based classification.

Overall, the Autoencoder demonstrates superior performance in identifying rare failure events, making it more suitable for predictive maintenance applications.

Model	Accuracy	Recall (Failure)	Remarks
MLP	High	Very Low	Biased toward majority class
LSTM	High	Very Low	No temporal advantage observed
Autoencoder	High	High	Effective anomaly detection

TABLE 8.1: Comparison of model performance

8.6 Key Observations

- **Impact of Class Imbalance:** The dataset is highly imbalanced, with failure cases forming a small minority. This significantly affects supervised models such as MLP and LSTM, which tend to bias predictions toward the majority class.
- **Limitations of Accuracy:** High accuracy observed in MLP and LSTM models is misleading, as it is driven by correct predictions of non-failure instances. These models fail to capture critical failure patterns.
- **Failure Detection is Critical:** In predictive maintenance, recall for failure cases is more important than overall accuracy. Missing a failure (false negative) can lead to severe operational and financial consequences.
- **Lack of Temporal Dependency:** The LSTM model does not show improvement over MLP, indicating that the dataset does not contain strong temporal relationships that can be exploited by sequence models.

- **Effectiveness of Autoencoder:** The Autoencoder effectively learns the normal behavior of the system and detects deviations using reconstruction error. This makes it highly robust to class imbalance and more suitable for anomaly detection tasks.
- **Clear Feature Separation:** The reconstruction error distribution shows a distinct separation between normal and anomalous samples, validating the effectiveness of the chosen thresholding approach.

8.7 Discussion and Future Improvements

The experimental results indicate that while deep learning models can learn general patterns, they struggle to detect rare failure events in imbalanced datasets.

To improve performance, the following approaches can be considered:

- Applying class weighting or oversampling techniques.
- Performing threshold tuning to improve recall for failure cases.
- Collecting more balanced datasets for training.

Chapter 9

Inferences/Insights Obtained/Difficulties faced

9.1 Model Performance Analysis

In this project, three deep learning models were implemented: a Multi-Layer Perceptron (MLP), a Long Short-Term Memory (LSTM) network, and an Autoencoder. The MLP and LSTM models were used for classification, while the Autoencoder was used for anomaly detection.

The MLP model performed consistently well and served as a strong baseline for classification. The LSTM model, although capable of capturing sequential dependencies, did not significantly outperform the MLP. This is primarily because the dataset does not exhibit true temporal characteristics.

The Autoencoder demonstrated effectiveness in detecting anomalies by identifying instances with high reconstruction error, which correspond to abnormal machine behavior.

9.2 Analysis of LSTM Performance

LSTM networks are designed for sequential or time-series data. In this project, sequences were artificially created using sliding windows to adapt the dataset for LSTM training. However, since the dataset does not inherently contain temporal dependencies, the LSTM model was unable to fully utilize its capabilities.

As a result, the performance of LSTM was comparable to or slightly lower than that of the MLP model.

9.3 Key Insights

- Machine failure is strongly influenced by specific failure modes such as tool wear failure (TWF), heat dissipation failure (HDF), power failure (PWF), overstrain failure (OSF), and random failure (RNF).
- Outliers in features such as rotational speed and torque indicate abnormal machine conditions and are important for predictive maintenance.
- Feature scaling using Z-score normalization significantly improved model convergence.
- Deep learning models are effective in capturing non-linear relationships in the dataset.

9.4 Difficulties Faced

9.4.1 Class Imbalance

The dataset exhibits class imbalance, where failure cases are significantly fewer than normal cases. This can lead to biased predictions toward the majority class and affect model performance.

9.4.2 Sequence Generation for LSTM

Since the dataset is not inherently sequential, artificial sequences were created using sliding windows. This approach may not accurately capture real-world temporal dependencies, limiting the effectiveness of the LSTM model.

9.4.3 Model Selection

Selecting appropriate models for both classification and anomaly detection required careful consideration of the dataset characteristics and problem objectives.

9.5 Conclusion

The MLP model proved to be effective for classification tasks, while the Autoencoder successfully handled anomaly detection. The LSTM model showed limited improvement due to the absence of true sequential data. These findings highlight the importance of aligning model selection with the nature of the dataset.

Chapter 10

Implementation Details and Resources

This project implementation, experiment tracking, and reproducibility resources are available through the following links:

- **GitHub Repository:**

https://github.com/kumaramardeep342/deep_learning_project

- **Weights & Biases Dashboard:**

<https://wandb.ai/kumaramardeep342-indian-institute-of-technology-madras/Predictive%20Maintenance%20for%20Industrial%20Milling%20Machines?nw=nwuserkumaramardeep342>

These resources provide complete access to the codebase, experimental configurations, and results, ensuring transparency and reproducibility of the proposed predictive maintenance system.

Bibliography

- [1] Stephan Matzka. Ai4i 2020 predictive maintenance dataset, 2020.